

myAgent开发系列

构建自己的AI Agent

AI技术专栏

2026年

目录

1. 先验知识与开发准备
2. 模块边界与通信机制
3. 技术选型：为什么用现在的技术栈
4. 项目初始化与目录结构
5. Bot数据模型设计
6. Prompt工程：让Bot有灵魂
7. Bot版本控制与发布
8. Bot执行引擎
9. 插件架构设计
10. 内置插件实现
11. 插件市场与发布
12. 插件沙箱与安全
13. 插件调试与日志

1. 先验知识与开发准备

myAgent开发系列 01

先验知识与开发准备

很多人用Coze/扣子、Dify，觉得好用。但好用背后的架构是怎么设计的？大多数文章只告诉你"怎么用"，没人告诉你"为什么这么设计"。

(参考OpenClaw进阶篇13)

核心观点：

不要急着写代码。在动手之前，先问自己：我真的准备好了吗？这一篇，我们先聊聊开发myAgent之前必须掌握的知识点，以及为什么这些知识比写代码更重要。

1

myAgent是什么？为什么要做这个项目？

myAgent = Self + Claw

。寓意：自我定制、自我进化的智能体平台。

OpenClaw（龙虾）是一个开源的AI Agent开发平台，功能强大，但部署门槛高、定制困难。

myAgent的目标是：做一个更轻量、更易定制、更适合学习的版本。

对比项

OpenClaw

myAgent

定位

企业级平台

学习+轻量定制

部署

Docker/K8s

本地即可运行

复度

高

中

学习曲线

陡峭

平缓

代码量

数十万行

几千行（核心）

myAgent不是要替代OpenClaw，而是提供一个更容易上手的学习版本。

学完之后，你再去研究OpenClaw的完整代码，会发现一切都是相通的。

2

开发myAgent之前，你需要掌握什么？

我见过太多人"只会调API"，不理解底层原理。这种人写的代码，

bug一堆，优化无从下手，遇到问题就抓瞎

。

在动手之前，你至少需要掌握这四个系列的知识：

系列

需要掌握的程度

为什么必须

一步一步学习OpenClaw

会用，理解核心概念

知道"成品"长什么样

一步一步学习AI Coding

能独立开发完整项目

前端后端都要自己写

Agent开发系列

理解Agent核心循环

这是整个系统的灵魂

本系列（myAgent开发）

从0到1构建平台

整合所有知识

3

技术栈选择：为什么用这些技术？

很多人问我：为什么选Next.js + Python？为什么不选XXX？

技术选型的核心原则是：够用就行，不要过度设计。

text

myAgent技术栈

前端

Next.js 14 (App Router)

React SSR框架，现代前端标配

TypeScript

类型安全，减少bug

TailwindCSS

快速样式开发

React Flow

工作流可视化

后端

Python 3.11+

AI领域首选，生态丰富

FastAPI

异步高性能，类型提示好

PostgreSQL

关系数据存储

Redis

Session和缓存

pgvector

向量检索（选pgvector因为一体化）

为什么不选XXX？

为什么不选Django？→ 太重，Agent场景不需要ORM的复功能

为什么不选Node.js后端？→ Python在AI领域生态碾压

为什么不选MongoDB？→ 结构化数据用PostgreSQL更

为什么不选Pinecone？→ pgvector够用，没必要多一个外部服务

4

最小知识集：Agent核心循环

如果你的时间只够学一个知识点，那必须是

Agent核心循环

。

(参考OpenClaw原理篇11)

python

Agent核心循环 – 这是一切的基础

def

agent_loop

(agent, user_input):

context = []

while

True

:

1. 思考：基于上下文，让LLM决定下一步

```
response = llm.
```

```
chat
```

```
(context + [user_input])
```

2. 如果是最终回答，结束

```
if
```

```
response.is_final:
```

```
return
```

```
response.content
```

3. 如果是工具调用，执行并把结果加回上下文

```
for
```

```
tool
```

```
in
```

```
response.tool_calls:
```

```
result =
```

```
execute_tool
```

```
(tool)
```

```
context.
```

```
append
```

```
(
```

```
f"Tool {tool.name} returned: {result}"
```

```
)
```

这个循环不理解清楚，后面的代码你根本看不懂。

很多人觉得"不就是调API吗"，但真正的问题是：什么时候该调用工具？如何让多轮对话保持连贯？遇到失败怎么办？

5

项目结构：一上来就要设计好

我见过太多"先写再说"的项目，最后代码成一团乱麻。

myAgent的项目结构，在第一天就定好了。

text

myAgent项目结构

selfclaw/

├── frontend/

Next.js前端

| ├── app/

| ├── components/

| └── ...

└── backend/

Python后端

| ├── api/

| ├── core/

Agent核心循环

| ├── plugins/

插件系统

| └── ...

└── shared/

共享类型定义

└── docker-compose.yml

一键启动

核心原则：前后端分离，共享类型定义。

这样前端改需求不会波及后端，后端升级不会搞挂前端。

6

准备好了吗？

如果上面的内容你都OK，那就可以开始了。如果有不懂的，

先去补对应系列的知识

。

记住：

磨刀不误砍柴工

。很多人急着写代码，结果写出来的代码自己都看不懂。花费一周做准备，比花费一个月填坑值多了。

总结：

myAgent是一个学习导向的AI Agent平台项目。开发它需要：1) 理解OpenClaw的核心概念 2) 掌握AI Coding的全栈开发能力 3)

理解Agent核心循环。技术栈选型原则是"够用就行"。项目结构在第一天就定好，避免后期重构。

有问题欢迎留言。

相关系列阅读

一步一步学习OpenClaw

一步一步学习AI Coding

Agent开发系列 01：从用户到创造者

下期预告：第2篇《myAgent核心架构解析》

myAgent开发系列 · 用代码理解AI

2. 模块边界与通信机制

myAgent开发系列 02

模块边界与通信机制

上一期我们讲了四层架构，但架构只是蓝图。

真正的问题是：这些层之间怎么通信？模块之间怎么解耦？

我见过太多"一开始写得爽，后来改一行代码崩整个系统"的惨案。问题出在哪？

模块边界没划清楚。

核心观点：

好的架构不是"怎么代码写得多漂亮"，而是"将来改需求的时候能不能只改一个地方"。模块边界划清楚，是一切的基础。

1

myAgent有哪几个核心模块？

拆开myAgent，内部其实就四个核心模块：

模块

职责

类比

Bot模块

Bot的创建、配置、版本管理

一个活的AI助手

插件模块

工具扩展、第三方能力接入

给助手装上各种技能

workflow模块

流程编排、多步骤任务执行

帮助手规划复任务

记忆模块

对话历史、知识存储、上下文

助手的大脑

这四个模块，

理论上可以独立开发、独立测试、独立部署

。但它们又不是完全隔离的——需要通信。

2

模块间通信：事件驱动架构

模块之间怎么通信？最笨的办法是直接调用：

python

耦合的写法 —— 不要这样做

class

Bot:

def

execute

(self):

memory = Memory()

直接创建

plugin = Plugin()

直接创建

workflow = Workflow()

直接创建

... 几十个直接调用

这种写法，短期爽，长期痛苦。某天你要把Memory换成另一个实现？

改到吐血。

正确做法是

事件驱动 + 依赖注入

:

python

解耦的写法 —— 事件总线模式

class

EventBus:

单例，全局事件总线

_subscribers: Dict[str, List[Callable]] = {}

def

subscribe

(self, event_type: str, handler: Callable):

if

event_type

not

self._subscribers:

self._subscribers[event_type] = []

self._subscribers[event_type].append(handler)

def

publish

(self, event: "Event"):

for

handler

in

self._subscribers.get(event.type, []):

handler(event)

事件驱动的核心思想是：

模块之间不直接调用，而是通过"发布事件"来通信

。谁关心谁订阅，不需要的模块完全无感知。

3

事件类型：哪些事件需要全局通信？

myAgent内部有这几类核心事件：

事件

触发时机

谁关心

bot.message

用户发消息

记忆模块、 workflow模块

bot.response

Bot生成回复

记忆模块、 插件模块

plugin.called

插件被调用

记忆模块、 监控模块

workflow.started

工作流启动

Bot模块、 监控模块

memory.stored

记忆被存储

监控模块

通过事件机制，

新增一个插件不需要改Bot代码

，只需要订阅它关心的事件。反过来，Bot代码重构，插件也完全不受影响。

4

模块内部：为什么不用的功能别乱加

说完模块间通信，再说说模块内部。

我见过最糟糕的设计是：在一个函数里塞了20个参数，做了15件不同的事。

这不是能力强，是缺乏边界意识。

每个模块应该有：

明确的输入

——这个函数需要什么？

明确的输出

——这个函数返回什么？

明确的职责

——这件事我只做，不做别的

python

好的设计：一个类只做一件事

class

BotConfig:

只负责Bot配置的管理

pass

class

BotExecutor:

只负责Bot的执行逻辑

pass

class

BotVersionManager:

只负责Bot的版本控制

pass

专业的人做专业的事，专业的类做专业的事。

一个Bot类干所有事，那是学生作业；生产级代码，一个类只干一件事。

5

实战：怎么画出模块边界图

很多人说"我知道要划分子模块，但不知道怎么画边界"。

我的经验是：

先写代码，后画边界

。不要想着一开始就把边界划清楚，这是不可能的。

正确流程是：

先写最核心的代码，让功能跑起来

观察哪些地方在频繁修改

把这些频繁修改的地方划成一个模块

定义好模块的接口（Interface）

后续修改只改模块内部，不影响其他模块

myAgent的模块边界不是我想出来的，是

根据实际开发过程中的改动频率总结出来的

。

总结：

模块边界清晰是架构设计的核心。模块间通信用事件驱动，模块内职责单一。不要过度设计，但也不要设计不足。最好的架构是改需求的时候只需要改一个地方。

有问题欢迎留言。

相关系列阅读

myAgent开发系列 01：先验知识与开发准备

Agent开发系列 21：消息总线 + 团队编排

下期预告：第3篇《技术选型：为什么用现在的技术栈》

3. 技术选型：为什么用现在的技术栈

myAgent开发系列 03

技术选型：为什么用现在的技术栈

每次技术分享会，都有人问我：为什么你们选这个技术？为什么不选那个？

今天系统性地解答一下。

技术选型不是选最新的，而是选最合适的。

脱离场景谈技术好坏，都是耍流氓。

核心观点：

技术选型的核心是"够用就行"。过度设计是浪费，不足设计是隐患。myAgent的目标是学习和轻量定制，所以选型的原则是：最小依赖、最快上手、最易调试。

1

前端：Next.js + TypeScript

先说前端。很多人问：为什么不用Vue？为什么不用纯React？

Next.js 14 (App Router)

是我目前最推荐的React框架。为什么？

SSR/SSG一把梭

——不管是做SEO页面还是做API动态页面，一套架构搞定

Server Components

——减少客户端JS，提升首屏加载速度

API Routes

——前后端可以放在一个项目里，学习成本最低

text

Next.js 14 项目结构

app/

└── page.tsx

页面组件 (Server Component)

└── api/

| └── bots/

API Routes

└── bots/

| └── [id]/

| └── page.tsx

Bot详情页

└── layout.tsx

根布局

TypeScript

是必选的。有人觉得TypeScript增加学习成本，但

减少的debug时间大于增加的学习时间

。尤其是团队协作时，类型即文档，看类型就知道函数该怎么用。

为什么不用Vue？Vue3的Composition API很好，但

Next.js的生态更成熟，AI相关的组件库支持更好

。比如Vercel官方出的AI SDK，就是为Next.js量身定制的。

2

后端：Python + FastAPI

后端选

Python 3.11+ + FastAPI

，这个组合在AI领域几乎是标配。

为什么是Python？

AI生态碾压一切

——LangChain、AutoGen、LlamaIndex，全是Python先支持

数据处理方便

——Agent场景大量JSON/文本处理，Python字典比Node.js对象方便太多

异步支持好

——Python 3.11+的asyncio已经非常成熟

为什么是FastAPI而不是Django或Flask？

text

FastAPI vs Django vs Flask

#

Django: 太重，ORM那套对Agent场景没用，自带admin后台根本用不上

Flask: 太轻，缺乏类型提示，写大项目容易变成面条代码

FastAPI: 刚刚好，类型提示完善，异步支持好，自动文档生成

FastAPI示例

from

fastapi

import

FastAPI

from

pydantic

import

BaseModel

class

BotCreate(BaseModel):

name: str

description: str |

None

None

自动生成OpenAPI文档，类型检查，异步支持 —— 全有了

FastAPI的

自动文档生成

功能简直是开发神器。打开/docs就能看到完整的API文档，还能直接在线调试。

后端写完，前端对着文档就能联调，不用专门开会对接口。

3

数据库：PostgreSQL + Redis

数据库选型是争议最大的。有人觉得PostgreSQL太重，有人觉得MongoDB更灵活。

我的观点：

结构化数据用PostgreSQL，缓存用Redis，向量检索用pgvector扩展

。

场景

选型

原因

用户、Bot配置

PostgreSQL

强类型、事务支持好

Session、缓存

Redis

高性能，TTL支持好

向量检索

pgvector

一体化，不需要额外服务

日志、监控

PostgreSQL

不需要专门搭ELK

为什么不用MongoDB？

MongoDB的JSON存储看起来灵活，但实际上

缺少强类型约束，数据质量难以保证

Agent场景的数据，其实大部分是有结构的——Bot配置、用户信息、对话记录，用PostgreSQL的JSONB字段完全够用

pgvector一体化

是加分项，别的数据库做不到

为什么不用Pinecone等专用向量数据库？

pgvector够用

——百万级向量完全没问题

少一个外部依赖

——部署复杂度降低一半

——一致性好

——所有数据在一个数据库，事务保证更强

4

工具链：Docker Compose一把梭

部署工具选Docker Compose，而不是直接上Kubernetes。

不是K8s不好，是大多数场景用不上。

yaml

docker-compose.yml

services:

frontend:

build: ./frontend

ports:

"3000:3000"

depends_on:

backend:

build: ./backend

ports:

"8000:8000"

environment:

postgres:

image: pgvector/pgvector:pg15

一个镜像同时跑PostgreSQL和向量检索

redis:

image: redis:7-alpine

docker-compose up

就能跑完整环境，

没有任何高深概念

。等项目大了，需要水平扩展了，再考迁移到K8s也不。

5

技术选型的反面教材

说完了为什么选这些，再说说常见反面教材。

反面教材1：追新党

"我要用最新的Rust做后端！"、"我要用最新发布的D1数据库！"

新技术当然好，但

新技术的坑你踩得起吗？

文档不完善、周边库少、出了问题Google搜不到答案。生产环境出问题，老板可不会听你解释"这是新技术嘛"。

反面教材2：全能党

"既然要微服务，那就拆20个Docker容器吧！"

一个Bot平台，你需要独立部署的容器有：frontend、backend、postgres、redis。够了。

非要把message queue拆出来、非要把logger拆出来、每个模块一个容器——这是过度设计。

反面教材3：权威党

"Google用K8s，所以我们也要用K8s！"、"字节跳动用Go，我们也要用Go！"

脱离场景谈技术，都是耍流氓。

Google的流量是每秒亿级，你的流量是每秒百级。抄架构不是抄作业，场景不同，方案完全不同。

总结：

myAgent的技术栈是：Next.js + TypeScript + FastAPI + PostgreSQL + Redis + pgvector + Docker Compose。选型原则是"够用就行，最小依赖"。不追新，不追大公司方案，只选最适合学习和小规模部署的组合。

有问题欢迎留言。

相关系列阅读

AI Coding系列 14：AI-Coding架构

myAgent开发系列 01：先验知识与开发准备

下期预告：第4篇《项目初始化与目录结构》

myAgent开发系列 · 用代码理解AI

4. 项目初始化与目录结构

myAgent开发系列 04

项目初始化与目录结构

技术选型定好了，接下来开始搭建项目。这一步很多人会忽略，但我认为

目录结构是架构设计的实体化

——你的代码组织方式，直接反映了你对系统的理解程度。

很多人喜欢在写代码之前就画一个完美的目录结构图。我觉得没必要，

先跑起来，再重构

。

核心观点：

目录结构不是一开始设计出来的，是随着系统复杂度增加，逐步演化出来的。但最初的结构要定好边界，给后期演化留足空间。

1

Monorepo架构：为什么前后端放一个仓库？

myAgent采用

Monorepo

架构——前后端代码放在同一个Git仓库里。

text

selfclaw/

|—— frontend/

Next.js 14 App Router

|—— backend/

Python FastAPI

|—— shared/

共享类型定义

├── docker-compose.yml

└── README.md

为什么不用两个独立仓库？

共享类型更方便

——TypeScript的类型定义可以直接给Python用

版本同步

——API改了，类型定义跟着改，不容易漏

一次clone搞定所有

——新人上手成本低

为什么不装NPM Workspace或者Python Poetry？

因为没必要。

这个项目规模还不到需要这些工具的时候。简单即美。

2

Shared包：共享类型定义

Shared目录是myAgent的核心设计之一。

前后端的类型定义，只在这里写一次。

text

shared/

├── types/

| ├── bot.ts

Bot相关类型

| ├── plugin.ts

插件相关类型

| └── workflow.ts

工作流相关类型

|—— python/

| |—— models.py

Pydantic模型（从types生成或手写）

|—— package.json

TypeScript编译

类型定义示例：

typescript

// shared/types/bot.ts

export

interface

BotConfig {

id: string;

name: string;

description: string;

model:

string

;

temperature: number;

maxTokens: number;

plugins: string[];

// 启用的插件ID列表

workflowId?: string;

// 关联的工作流

status:

'draft'

|

```
'published'

;

}

export

interface

BotMessage {

id: string;

botId: string;

role:

'user'

|

'assistant'

|

'system'

;

content: string;

createdAt: string;

}
```

类型即文档

。前端看到`BotConfig`就知道Bot需要哪些字段，后端看到就知道API要返回什么格式。这就是共享类型的价值。

3

Frontend目录：Next.js App Router

前端用Next.js 14的App Router，这是目前React SSR的最新方案。

text

frontend/

└── app/

| |—— page.tsx

首页

| |—— layout.tsx

根布局

| |—— bots/

| | |—— page.tsx

Bot列表页

| | |—— new/page.tsx

创建Bot

| | |—— [id]/

| | |—— page.tsx

Bot详情

| | |—— edit/page.tsx

编辑Bot

| |—— api/

| |—— bots/

| |—— route.ts

API Routes

|—— components/

| |—— ui/

基础UI组件

| |—— bot/

Bot相关组件

| |—— workflow/

工作流相关

|—— lib/

| └── api.ts

API调用封装

|─── package.json

└── next.config.js

App Router的核心是

约定式路由

◦ 文件路径就是路由，不需要额外配置。

4

Backend目录：FastAPI分层

后端采用

分层架构

：API层 → Service层 → Repository层。

text

backend/

|─── api/

| └── bots.py

Bot相关API

| └── plugins.py

插件相关API

| └── workflows.py

工作流相关API

|─── core/

| └── agent.py

Agent核心循环

| └── event_bus.py

事件总线

| └── config.py

配置管理

|── services/

| |── bot_service.py

Bot业务逻辑

| └── memory_service.py

记忆业务逻辑

|── repositories/

| |── bot_repo.py

Bot数据访问

| └── message_repo.py

消息数据访问

|── models/

| └── database.py

SQLAlchemy模型

|── main.py

└── requirements.txt

为什么分三层？

API层

——只处理请求解析和响应格式化，不写业务逻辑

Service层

——写业务逻辑，调用多个Repository或服务

Repository层

——只管数据库操作，SQL写这里

这种分层的好处是

可测试性极强

◦ Service层可以完全脱离数据库单独测试，Mock掉Repository就行。

5

初始化脚本：一键启动

项目根目录的

init.sh

脚本负责初始化开发环境：

bash

#!/bin/bash

init.sh – 一键初始化开发环境

1. 安装前端依赖

cd frontend && npm install && cd ..

2. 安装后端依赖

cd backend && pip install -r requirements.txt && cd ..

3. 启动数据库服务

docker-compose up -d postgres redis

4. 初始化数据库

cd backend && python -m alembic upgrade head && cd ..

echo

"初始化完成！运行 docker-compose up 启动服务"

新同事clone代码后，执行一个脚本就能开始开发。

这就是好架构的体现——降低协作成本。

总结：

myAgent采用Monorepo架构，前端Next.js、后端FastAPI，通过Shared包共享类型。目录结构是系统理解的实体化——先跑起来，再逐步优化。

有问题欢迎留言。

相关系列阅读

AI Coding系列 16：项目结构设计

myAgent开发系列 03：技术选型

下期预告：第5篇《Bot数据模型设计》

myAgent开发系列 · 用代码理解AI

5. Bot数据模型设计

myAgent开发系列 05

Bot数据模型设计

进入Bot核心篇。这一篇，我们来聊一个看似简单、实则极其重要的问题：

Bot的数据模型应该怎么设计？

很多人觉得Bot不就是"名字+描述+Prompt"吗？但当你真正去做生产级应用，会发现

配置比想象复

——版本管理、变量插值、插件关联、 workflow 绑定.....每一个都是坑。

核心观点：

Bot数据模型是整个myAgent的核心。一切后续功能——插件、workflow、记忆——都建立在Bot模型之上。模型设计不好，后续扩展全是噩梦。

1

Bot实体：最基础的配置

先来看Bot的最基础实体：

```
python
```

```
# backend/models/bot.py
```

```
from
```

```
sqlmodel
```

```
import
```

```
SQLModel, Field
```

```
from
```

```
typing
```

```
import
```

```
Optional
```

```
from
datetime

import
datetime

class
Bot(SQLModel, table=True):

    __tablename__ =

    "bots"

    id: str = Field(primary_key=True, default=

    lambda

    : uuid.uuid4().hex)

    name: str = Field(max_length=100)

    # Bot名称

    description: str = Field(default=

    ""

    )

    # 描述

    avatar: Optional[str] =

    None

    # 头像URL

    model: str = Field(default=

    "gpt-4"

    )

    # 使用的模型

    temperature: float = Field(default=0.7)

    # 温度参数
```

```
max_tokens: int = Field(default=2000)

# 最大token

system_prompt: str = Field(default=

"""

)

# 系统提示词

prompt_config: dict = Field(default=dict)

# Prompt变量配置

plugin_ids: list[str] = Field(default=list)

# 启用的插件

workflow_id: Optional[str] =

None

# 关联的工作流

status: str = Field(default=

"draft"

)

# draft | published

version: int = Field(default=1)

# 当前版本号

owner_id: str = Field(index=True)

# 拥有者

tenant_id: str = Field(index=True)

# 租户ID

created_at: datetime = Field(default=datetime.utcnow)

updated_at: datetime = Field(default=datetime.utcnow)
```

这个模型有几个关键设计：

prompt_config

——这是个JSON字段，存Prompt里的变量配置。比如Prompt里有个

```
{"variable": "name", "default": "用户", "type": "string"}
```

，运行时会被替换

plugin_ids

——Bot启用的插件列表，不是直接存储插件对象，而是存ID，便于解耦

status

——draft/published状态机，支持先编辑再发布

2

Bot版本：支持灰度发布和回滚

生产级Bot必须有版本控制。为什么？

你发布了一个新版本，发现有bug，需要

快速回滚

你想

灰度发布

，先让10%用户用新版本

你需要

对比两个版本

的效果差异

python

class

BotVersion(SQLModel, table=True):

__tablename__ =

"bot_versions"

id: str = Field(primary_key=True, default=

lambda

```
: uuid.uuid4().hex)
```

```
bot_id: str = Field(foreign_key=
```

```
"bots.id"
```

```
, index=True)
```

```
version: int = Field(index=True)
```

```
# 快照：发布时的完整配置
```

```
config_snapshot: dict = Field()
```

```
# 发布信息
```

```
status: str = Field(default=
```

```
"draft"
```

```
)
```

```
# draft | published | deprecated
```

```
rollout_percent: int = Field(default=100)
```

```
# 灰度发布比例
```

```
changelog: str = Field(default=
```

```
""
```

```
)
```

```
# 变更说明
```

```
published_at: Optional[datetime] =
```

```
None
```

```
published_by: Optional[str] =
```

```
None
```

核心设计：config_snapshot存发布时刻的完整配置快照

。这样回滚时直接用快照恢复，不需要计算diff。

3

Prompt配置Schema：让Prompt可编程化

上一节提到prompt_config，这里详细说说。

简单Bot只有一个system_prompt就够了，但

企业级Bot通常需要变量

:

text

一个客服Bot的Prompt

你是一个{company_name}的客服助手。

公司产品：{product_list}

服务时间：{service_hours}

当前用户：{user_name}

{xxx}都是变量，需要在prompt_config里定义

prompt_config的Schema设计：

python

prompt_config的JSON结构

prompt_config = {

"variables"

: [

{

"name"

:

"company_name"

,

"type"

:

"string"

,

"default"

:

"某公司"

,

"description"

:

"公司名称"

,

},

{

"name"

:

"product_list"

,

"type"

:

"string"

,

"source"

:

"knowledge_base"

,

从知识库动态获取

"kb_id"

:

"kb_xxx"

```
,  
}  
]  
}
```

这种设计的优势：

Prompt模板和具体值分离

- 同一个Prompt模板，可以给不同租户使用，只需要替换变量值。

4

对话会话：多轮对话的存储

Bot的另一个核心模型是会话（Session）和消息（Message）：

python

class

Conversation(SQLModel, table=True):

__tablename__ =

"conversations"

id: str = Field(primary_key=True)

bot_id: str = Field(foreign_key=

"bots.id"

, index=True)

user_id: str = Field(index=True)

bot_version_id: str = Field(index=True)

使用哪个版本

状态

status: str = Field(default=

"active"

)

active | closed

Token统计（用于计费）

total_tokens: int = Field(default=0)

created_at: datetime = Field(default=datetime.utcnow)

updated_at: datetime = Field(default=datetime.utcnow)

class

Message(SQLModel, table=True):

__tablename__ =

"messages"

id: str = Field(primary_key=True)

conversation_id: str = Field(foreign_key=

"conversations.id"

, index=True)

role: str

user | assistant | system | tool

content: str

Token统计

tokens: int = Field(default=0)

工具调用信息（如果role=tool）

tool_call_id: Optional[str] =

None

tool_name: Optional[str] =

None

created_at: datetime = Field(default=datetime.utcnow)

几个关键设计点：

bot_version_id

——记录对话使用的是哪个Bot版本，方便追溯和计费

total_tokens

——会话级别的Token统计，用于计费和监控

tool_call_id/tool_name

——支持工具调用的完整链路追溯

总结：

Bot数据模型是myAgent的核心。Bot模型存配置，BotVersion存快照支持回滚，prompt_config支持变量插值，Conversation/Message支持多轮对话。模型设计要考：可版本化、可变量化、可追溯。

有问题欢迎留言。

相关系列阅读

Agent开发系列 03：最小可行Agent

OpenClaw原理篇11：核心循环

myAgent开发系列 04：项目初始化

下期预告：第6篇《Prompt工程：让Bot有灵魂》

myAgent开发系列 · 用代码理解AI

6. Prompt工程：让Bot有灵魂

myAgent开发系列 06

Prompt工程：让Bot有灵魂

数据模型是骨架，Prompt是灵魂。同一个Bot模型，换一个Prompt，效果可能天差地别。

很多人觉得Prompt就是"随便写几句提示词"。

大错特错。

Prompt工程是AI时代最重要的技能之一。

核心观点：

Prompt不是越详细越好，也不是越简洁越好。好的Prompt是有结构的、有层次的、能被模型理解的。Prompt工程的核心是"把你脑子里的东西，准确传达给AI"。

1

Prompt的四个层次

一个完整的Prompt，通常包含这四个层次：

text

第一层：角色定义

你是一个专业的Python后端工程师。

第二层：任务定义

你的职责是帮助用户优化代码、解释技术方案、review代码。

第三层：约束条件

1. 回答必须基于Python最佳实践

2. 代码必须包含类型注解

3. 复问题必须先分析再给出方案

第四层：输出格式

用Markdown格式回答，代码块要标注语言。

层层递进，缺一不可。

没有角色定义，模型不知道以什么身份回答；没有任务定义，不知道要干什么；没有约束条件，可能跑偏；没有输出格式，可能给你一段没有标点的文字。

2

myAgent的Prompt模板系统

上一节讲了Bot数据模型里的prompt_config。这节课讲具体怎么实现变量替换。

python

```
# backend/core/prompt.py
```

```
import
```

```
re
```

```
from
```

```
typing
```

```
import
```

```
Dict, Any
```

```
class
```

```
PromptTemplate:
```

```
# Prompt模板类，负责变量插值
```

```
def
```

```
__init__
```

```
(self, template: str, config: Dict[str, Any]):
```

```
self.template = template
```

```
self.config = config
```

```
def
```

```
render
```

```
(self, context: Dict[str, Any]) -> str:
```

```
# 用正则匹配 {variable_name} 格式的变量
```

```
result = self.template
```

```
for
match

in

re.finditer(

r'\{(\w+)\}'

, self.template):

var_name = match.group(1)

# 优先级 : context > config > default

if

var_name

in

context:

value = context[var_name]

elif

var_name

in

self.config:

value = self.config[var_name]

else

:

value =

f'{{{var_name}}}'

# 保留原样

result = result.replace(

f'{{{var_name}}}'

, str(value))
```

```
return
```

```
result
```

使用示例：

```
python
```

```
# 使用示例
```

```
template =
```

```
"你是一个{bot_role}，服务于{company_name}公司。"
```

```
config = {
```

```
"bot_role"
```

```
:
```

```
"客服助手"
```

```
,
```

```
"company_name"
```

```
:
```

```
"某科技有限公司"
```

```
}
```

```
prompt = PromptTemplate(template, config)
```

```
print(prompt.render({}))
```

```
# 输出：你是一个客服助手，服务于某科技有限公司公司。
```

```
# 运行时覆盖
```

```
print(prompt.render({
```

```
"company_name"
```

```
:
```

```
"新公司"
```

```
}))
```

```
# 输出：你是一个客服助手，服务于新公司。
```

Few-shot示例管理

有时候，纯文字的Prompt不够用，需要

示例

来教模型什么是好的回答。

python

class

Example:

单个示例

input: str

output: str

class

ExampleBank:

示例库，支持动态选择

examples: List[Example] = []

def

select

(self, context: dict, k: int = 2) -> List[Example]:

根据上下文选择最相关的k个示例

可以用Embedding相似度，也可以用规则

return

self.examples[:k]

def

to_prompt

(self, examples: List[Example]) -> str:

转换为Prompt字符串

```
lines = []

for
ex
in
examples:

lines.append(

f"输入：

{ex.input}

"

)

lines.append(

f"输出：

{ex.output}

"

)

return

"

\n

"

.join(lines)
```

示例管理的关键是

按需选择

。不是把所有示例都塞进Prompt，而是根据当前对话的上下文，选择最相关的几个。

4

Prompt版本管理：不只是改文字

很多人忽略了Prompt的版本管理。但Prompt的每一次改动，都可能影响Bot的行为。

python

class

PromptVersion(SQLModel, table=True):

__tablename__ =

"prompt_versions"

id: str = Field(primary_key=True)

bot_id: str = Field(foreign_key=

"bots.id"

)

version: int

system_prompt: str

prompt_config: dict

examples: List[Example] = Field(default=list)

效果指标

avg_rating: float = Field(default=0.0)

用户平均评分

usage_count: int = Field(default=0)

使用次数

created_at: datetime

created_by: str

changelog: str =

"""

Prompt不是一次性写完就不管了，而是要持续优化。

通过版本管理和效果追踪，可以量化每个Prompt版本的好坏。

总结：

Prompt是Bot的灵魂。好的Prompt有四个层次：角色、任务、约束、格式。myAgent通过PromptTemplate支持变量插值，通过ExampleBank支持Few-shot，通过PromptVersion支持版本追踪。Prompt工程是AI时代最重要的技能。

有问题欢迎留言。

相关系列阅读

AI Coding系列 04：提示词技巧

Agent开发系列 05：感知

myAgent开发系列 05：Bot数据模型

下期预告：第7篇《Bot版本控制与发布》

myAgent开发系列 · 用代码理解AI

7. Bot版本控制与发布

myAgent开发系列 07

Bot版本控制与发布

上一节讲了Bot数据模型和Prompt工程。这一节，讲一个生产级Bot必须具备的能力：

版本控制和发布管理

。

很多人觉得"改完Prompt直接发布就行了"。但生产环境中，你可能需要：凌晨三点发现线上Bot有问题要回滚；想让10%用户先试试新版本；想对比新旧两个版本的效果差异。

没有版本控制的Bot，就是一颗定时炸弹。

核心观点：

版本控制不只是"记录改了啥"，而是提供"回到过去"和"选择未来"的能力。Draft/Published状态机 + 灰度发布，是生产级Bot的标配。

1

状态机：Draft和Published

Bot的基本状态只有两个：

Figure: Bot State Machine – Draft and Published states

Draft（草稿）

——可以随便改，不影响线上用户

Published（已发布）

——正在服务用户的版本，不能随便改

python

```
# backend/services/bot_service.py
```

```
class
```

```
BotService:
```

```

def
create_bot

(self, bot_data: BotCreate) -> Bot:

# 新创建的Bot默认是Draft

bot = Bot(

name=bot_data.name,

description=bot_data.description,

status=

"draft"

,

# 默认草稿

version=1,

owner_id=bot_data.owner_id,

tenant_id=bot_data.tenant_id,

)

db.add(bot)

return

bot

def

publish_bot

(self, bot_id: str, changelog: str =

""

) -> BotVersion:

bot = self.get_bot(bot_id)

if

bot.status ==

```

```
"published"
```

```
:
```

```
# 已经发布的Bot，需要创建新版本
```

```
return
```

```
self._create_new_version(bot, changelog)
```

```
else
```

```
:
```

```
# 草稿状态，直接发布第一个版本
```

```
return
```

```
self._publish_first_version(bot, changelog)
```

```
2
```

版本快照：发布即存档

发布时最重要的操作是

创建版本快照

——把当前配置完整保存下来。

```
python
```

```
def
```

```
_create_version_snapshot
```

```
(bot: Bot) -> BotVersion:
```

```
# 创建配置快照
```

```
snapshot = {
```

```
"name"
```

```
: bot.name,
```

```
"model"
```

```
: bot.model,
```

```
"temperature"
```

```
: bot.temperature,

"max_tokens"

: bot.max_tokens,

"system_prompt"

: bot.system_prompt,

"prompt_config"

: bot.prompt_config,

"plugin_ids"

: bot.plugin_ids,

"workflow_id"

: bot.workflow_id,

}

version = BotVersion(

bot_id=bot.id,

version=bot.version,

config_snapshot=snapshot,

status=

"published"

,

published_at=datetime.utcnow(),

)

db.add(version)

# 更新Bot状态

bot.status =

"published"

bot.published_version_id = version.id
```

return

version

快照的好处是：即使Bot配置被改的面目全非，也能一键回到任何一个历史版本。

3

回滚：让事故损失最小化

版本控制的核心价值之一是

回滚能力

。

python

def

rollback_bot

(self, bot_id: str, target_version: int) -> Bot:

bot = self.get_bot(bot_id)

查找目标版本

version = db.query(BotVersion).filter(

BotVersion.bot_id == bot_id,

BotVersion.version == target_version

).first()

if

not

version:

raise

ValueError(

f"Version {target_version} not found"

)

从快照恢复配置

```
snapshot = version.config_snapshot
```

```
bot.model = snapshot[
```

```
"model"
```

```
]
```

```
bot.temperature = snapshot[
```

```
"temperature"
```

```
]
```

```
bot.system_prompt = snapshot[
```

```
"system_prompt"
```

```
]
```

```
# ... 其他字段
```

```
# 创建新的回滚版本记录
```

```
bot.version += 1
```

```
self._create_version_snapshot(bot)
```

```
return
```

```
bot
```

回滚操作的关键是：

不修改历史快照，而是基于快照创建新版本

。这样任何版本都可以追溯。

4

灰度发布：小心驶得万年船

生产级发布不只是"全量"或"不回滚"，还有

灰度发布

。

python

def


```
publish_with_rollout
```

```
(self, bot_id: str, rollout_percent: int = 10):
```

```
# rollout_percent: 0-100，0就是不发布，100就是全量
```

```
if
```

```
rollout_percent == 0:
```

```
return
```

```
# 不发布
```

```
bot = self.get_bot(bot_id)
```

```
new_version = self._create_new_version(bot)
```

```
# 设置灰度比例
```

```
new_version.rollout_percent = rollout_percent
```

```
new_version.status =
```

```
"published"
```

```
if
```

```
rollout_percent == 100:
```

```
# 全量发布，把旧版本标记为deprecated
```

```
self._deprecate_old_versions(bot.id, new_version.version)
```

```
else
```

```
:
```

```
# 灰度发布，新旧版本共存
```

```
pass
```

灰度发布时，需要根据用户ID做路由：

```
python
```

```
def
```

```
get_bot_version_for_user
```

```
(bot_id: str, user_id: str) -> BotVersion:
```

```
versions = db.query(BotVersion).filter(

BotVersion.bot_id == bot_id,

BotVersion.status ==

"published"

).all()

# 根据user_id的hash决定用哪个版本

user_hash = hash(user_id) % 100

for

v

in

versions:

if

v.rollout_percent > user_hash:

return

v

# 默认返回最新全量版本

return

self._get_latest_full_version(versions)
```

灰度发布的核心是"用户无感知"——同一个Bot，对不同用户可能返回不同版本。

总结：

Bot版本控制包含状态机（Draft/Published）、版本快照、回滚能力、灰度发布。生产级Bot必须具备这些能力，才能应对凌晨三点的线上事故。

有问题欢迎留言。

相关系列阅读

Agent开发系列 19：动态调整

myAgent开发系列 05：Bot数据模型

下期预告：第8篇《Bot执行引擎》

myAgent开发系列 · 用代码理解AI

8. Bot执行引擎

myAgent开发系列 08

Bot执行引擎

数据模型是静态的，Prompt是配置，执行引擎才是动态的

——一个请求进来，Bot是怎么处理的？

执行引擎是myAgent最核心的模块。它负责：接收请求、调用记忆、调用LLM、处理工具调用、返回响应。

所有其他模块——插件、工作流、记忆——都围绕执行引擎协作。

核心观点：

执行引擎的核心是"编排"——不是自己干所有事，而是协调各个模块各司其职。好的执行引擎是"我在调度，不是在干活"。

1

执行流程：请求到响应的旅程

一个用户消息到达Bot执行引擎，完整的处理流程是这样的：

Figure: Bot Execution Flow – From request to response

text

Bot执行流程

用户消息

↓

1. 会话管理：查找或创建会话

↓

2. 记忆召回：从记忆中获取相关上下文

↓

3. Prompt渲染：用变量和记忆组装完整Prompt

↓

4. LLM调用：发送到LLM获取响应

↓

5. 工具处理：如果需要工具调用，执行工具

↓

6. 记忆存储：把本次对话存入记忆

↓

7. 返回响应

这个流程看起来简单，但每一步都有坑。

2

会话管理：找到属于你的上下文

第一步是会话管理。用户发起请求，系统要找到这个用户的会话上下文。

python

class

SessionManager:

def

get_or_create_session

(self, bot_id: str, user_id: str) -> Conversation:

conv = db.query(Conversation).filter(

Conversation.bot_id == bot_id,

Conversation.user_id == user_id,

Conversation.status ==

"active"

).first()

if

not

conv:

```
conv = Conversation(

bot_id=bot_id,

user_id=user_id,

bot_version_id=self._get_bot_version_id(bot_id),

)

db.add(conv)

return

conv
```

会话管理的关键设计：

一个用户对一个Bot只有一个活跃会话

◦ 这样简化了上下文管理的复杂度。

3

消息历史：组装完整的对话上下文

拿到会话后，需要获取历史消息，组装成完整的对话上下文。

python

async def

build_messages

(self, conv: Conversation) -> List[Dict]:

messages = []

1. 系统Prompt

bot = self.bot_service.get_bot(conv.bot_id)

system_prompt = self._render_system_prompt(bot)

messages.append({

"role"

:

"system"

,

"content"

: system_prompt

})

2. 历史消息（限制条数，避免超出Token限制）

history = self.message_repo.get_recent(

conv.id,

limit=self._get_max_history(bot.model)

)

messages.extend([

"role"

: m.role,

"content"

: m.content

}]

for

m

in

history])

return

messages

历史消息不能无限制存储

，因为LLM有Token上限。需要根据模型的最大上下文长度，动态计算能存多少条历史消息。

4

限流控制：保护你的LLM调用

执行引擎必须有限流能力。不然用户刷接口，你的API配额分分钟用完。

```
python

from

redis

import

Redis

class

RateLimiter:

    def

    __init__

    (self, redis_client: Redis):

    self.redis = redis_client

    async def

    check_rate_limit

    (self, user_id: str, bot_id: str) -> bool:

    key =

    f"rate_limit:{user_id}:{bot_id}"

    # 滑动窗口限流：每分钟最多N次

    count = await self.redis.get(key)

    if

    count

    and

    int(count) >= 60:

    return

    False

    # 超限

    await
```



```
self.redis.incr(key)
```

```
await
```

```
self.redis.expire(key, 60)
```

```
# 60秒窗口
```

```
return
```

```
True
```

限流是保护系统定性的最后一道防线

◦ Redis的滑动窗口算法简单高效，能应对大多数限流场景。

总结：

Bot执行引擎负责完整的请求处理流程：会话管理→消息历史→Prompt渲染→LLM调用→工具处理→记忆存储→响应返回。执行引擎是编排者，不是执行者。限流保护系统不被冲垮。

有问题欢迎留言。

相关系列阅读

Agent开发系列 17：执行引擎

myAgent开发系列 07：Bot版本控制

下期预告：架构设计篇完结，接下来进入Bot核心篇

myAgent开发系列 · 用代码理解AI

9. 插件架构设计

myAgent开发系列 09

插件架构设计

进入插件系统篇。

插件系统是myAgent最强大的能力之一——让Bot可以像人一样"学会"各种新技能。

很多人用过OpenClaw的插件，觉得很强大。但它背后是怎么实现的？这一篇，我们来实现一个灵活的插件框架。

(参考Agent开发系列13：Tool接口设计)

核心观点：

插件系统的核心是"接口抽象"——只要符合接口，任何人都能开发插件。好的插件系统，应该是"你只管写插件逻辑，剩下的交给框架"。

1

插件接口：标准化是解耦的关键

先定义插件接口。

(Tool接口设计详见Agent开发系列13)

python

```
# backend/plugins/base.py
```

```
from
```

```
abc
```

```
import
```

```
ABC, abstractmethod
```

```
from
```

```
typing
```

```
import
```

Any, Dict

from

pydantic

import

BaseModel

class

ToolParameter(BaseModel):

name: str

type: str

description: str

required: bool =

True

default: Any =

None

class

ToolDefinition(BaseModel):

name: str

description: str

parameters: list[ToolParameter] = []

class

ToolResult(BaseModel):

success: bool

result: Any =

None

error: str =

None

```

class
BaseTool(ABC):
# 插件基类，所有插件必须继承这个类

@property

@abstractmethod

def

definition

(self) -> ToolDefinition:

# 返回插件定义，供LLM理解插件能力

pass

@abstractmethod

async def

execute

(self, **kwargs) -> ToolResult:

# 执行插件逻辑

pass

```

核心设计：definition告诉LLM这个插件有什么用，execute才是真正的逻辑。

2

插件注册：动态发现与加载

插件写好了，怎么让系统知道？用插件注册表。

```

python

# backend/plugins/registry.py

class

ToolRegistry:

_tools: Dict[str, BaseTool] = {}

@classmethod

```

```
def
register

(cls, tool: BaseTool):

definition = tool.definition

cls._tools[definition.name] = tool

@classmethod

def

get

(cls, name: str) -> BaseTool |

None

:

return

cls._tools.get(name)

@classmethod

def

list_all

(cls) -> List[ToolDefinition]:

return

[tool.definition

for

tool

in

cls._tools.values()]

# 用装饰器注册插件

def

register_tool
```

```
(tool_class):
```

```
ToolRegistry.
```

```
register
```

```
(tool_class())
```

```
return
```

```
tool_class
```

使用装饰器，插件开发者只需要一行代码就能注册：

```
python
```

```
# backend/plugins/search.py
```

```
from
```

```
.base
```

```
import
```

```
BaseTool, ToolDefinition, ToolResult
```

```
# 一行装饰器，自动注册
```

```
@register_tool
```

```
class
```

```
SearchTool(BaseTool):
```

```
@property
```

```
def
```

```
definition
```

```
(self):
```

```
return
```

```
ToolDefinition(
```

```
name=
```

```
"search"
```

```
,
```

```
description=
```

```
"搜索互联网获取信息"
```

```
,
```

```
parameters=[{
```

```
"name"
```

```
:
```

```
"query"
```

```
,
```

```
"type"
```

```
:
```

```
"string"
```

```
,
```

```
"description"
```

```
:
```

```
"搜索关键词"
```

```
}]
```

```
)
```

```
async def
```

```
execute
```

```
(self, query: str) -> ToolResult:
```

```
try
```

```
:
```

```
results = await search_api.
```

```
search
```

```
(query)
```

```
return
```

```
ToolResult(success=
```

```
True
```

```
, result=results)
```

```
except
```

```
Exception
```

```
as
```

```
e:
```

```
return
```

```
ToolResult(success=
```

```
False
```

```
, error=
```

```
str
```

```
(e))
```

```
3
```

插件生命周期

每个插件都有自己的生命周期：

```
python
```

```
class
```

```
BaseTool(ABC):
```

```
# 可选：插件初始化
```

```
async def
```

```
on_load
```

```
(self):
```

```
"插件加载时调用，可以做初始化操作"
```

```
pass
```

```
# 可选：插件卸载前调用
```



```
async def
```

```
on_unload
```

```
(self):
```

```
"插件卸载时调用，可以做清理操作"
```

```
pass
```

```
# 可选：每次执行前调用
```

```
async def
```

```
before_execute
```

```
(self, **kwargs):
```

```
"执行前校验，比如检查API配额"
```

```
pass
```

```
# 可选：每次执行后调用
```

```
async def
```

```
after_execute
```

```
(self, result: ToolResult):
```

```
"执行后处理，比如记录日志"
```

```
pass
```

生命周期钩子让插件可以做一些横切关注点

——比如日志、性能监控、错误追踪。

4

Bot与插件的绑定

Bot启用哪些插件？在Bot模型里定义。

(详见myAgent开发系列05：Bot数据模型)

```
python
```

```
# Bot启用哪些插件，由plugin_ids决定
```

```
class
```

BotExecutor:

@property

def

available_tools

(self):

bot = self.bot

return

[

ToolRegistry.

get

(plugin_id)

for

plugin_id

in

bot.plugin_ids

if

ToolRegistry.

get

(plugin_id)

]

这就是解耦的好处：

Bot只存储插件ID，具体插件实现在运行时才绑定

。改插件实现不需要改Bot配置。

总结：

插件系统的核心是接口抽象和动态注册。BaseTool定义接口规范，ToolRegistry负责插件注册，Bot通过plugin_ids引用插件。生命周期钩子让插件可以处理横切关注点。

有问题欢迎留言。

相关系列阅读

Agent开发系列 13：Tool接口设计

Agent开发系列 15：Function Calling + 工具安全

myAgent开发系列 05：Bot数据模型

下期预告：第10篇《内置插件实现》

myAgent开发系列 · 用代码理解AI

10. 内置插件实现

myAgent开发系列 10

内置插件实现

上一讲讲了插件架构，这一讲来实现几个内置插件。

(详见Agent开发系列14：内置工具实现)

myAgent内置几个最常用的插件：Search（搜索）、DALLE（绘图）、Browser（浏览器）。这些插件覆盖了大部分日常场景。

核心观点：

内置插件要"够用但不臃肿"。每个插件只做一件事，但把这件事做到极致。

1

Search插件：让Bot能上网

python

import

httpx

from

.base

import

BaseTool, ToolDefinition, ToolResult

@register_tool

class

SearchTool(BaseTool):

@property

def

definition

(self) -> ToolDefinition:

return

ToolDefinition(

name=

"search"

,

description=

"搜索互联网获取实时信息"

,

parameters=[{

"name"

:

"query"

,

"type"

:

"string"

,

"description"

:

"搜索关键词，最多50字"

,

"required"

:

True

}]

)

async def

execute

(self, query: str) -> ToolResult:

try

:

async

with

httpx.AsyncClient()

as

client:

response =

await

client.

POST

(

"https://api.search.example.com/search"

,

json={

"query"

: query,

"limit"

:

5

},

timeout=

10.0

)

results = response.

json

()

return

ToolResult(success=

True

, result=results)

except

Exception

as

e:

return

ToolResult(success=

False

, error=

str

(e))

2

DALLE插件：让Bot能画画

python

@register_tool

class

DALLETotem(BaseTool):

@property

```
def
definition
(self):
return
ToolDefinition(
name=
"generate_image"
,
description=
"根据描述生成图片"
,
parameters=[{
"name"
:
"prompt"
,
"type"
:
"string"
,
"description"
:
"图片描述，英文最佳"
,
"required"
:
```


True

}, {

"name"

:

"size"

,

"type"

:

"string"

,

"description"

:

"图片尺寸：256x256, 512x512, 1024x1024"

,

"required"

:

False

,

"default"

:

"1024x1024"

}]

)

async def

execute

(self, prompt: str, size: str =

"1024x1024"

) -> ToolResult:

response =

await

openai.Image.

acreate

(

prompt=prompt,

size=size

)

image_url = response.data[

0

].url

return

ToolResult(success=

True

, result={

"url"

: image_url})

3

Browser插件：让Bot能浏览网页

python

@register_tool

class

BrowserTool(BaseTool):

@property

```
def
definition
(self):
return
ToolDefinition(
name=
"browser"
,
description=
"打开网页并提取内容"
,
parameters=[{
"name"
:
"url"
,
"type"
:
"string"
,
"description"
:
"要访问的网页URL"
,
"required"
```

```
True

}]

)

async def

execute

(self, url: str) -> ToolResult:

try

:

content =

await

self._fetch_page(url)

return

ToolResult(success=

True

, result={

"content"

: content})

except

Exception

as

e:

return

ToolResult(success=

False

, error=

str
```

(e))

```
async def
```

```
_fetch_page
```

```
(self, url: str) -> str:
```

```
    async
```

```
    with
```

```
        httpx.AsyncClient()
```

```
        as
```

```
        client:
```

```
        response =
```

```
            await
```

```
                client.
```

```
                    get
```

```
                        (url, timeout=
```

```
                            30.0
```

```
                        )
```

```
                    return
```

```
                        self._extract_text(response.text)
```

```
                    4
```

插件参数校验

参数校验是插件安全的重要一环。

(详见Agent开发系列15：工具安全)

```
python
```

```
def
```

```
    validate_parameters
```

```
(definition: ToolDefinition, params: dict) -> tuple[bool, str]:
```

```
# 检查必填参数
```

```
for
```

```
param
```

```
in
```

```
definition.parameters:
```

```
if
```

```
param.required
```

```
and
```

```
param.name
```

```
not
```

```
in
```

```
params:
```

```
return
```

```
False
```

```
,
```

```
f
```

```
"Missing required parameter:
```

```
{param.name}
```

```
"
```

```
# 检查类型
```

```
for
```

```
name, value
```

```
in
```

```
params.items():
```

```
param_def =
```

```
next
```

```
((p
for
p
in
definition.parameters
if
p.name == name),
None
)
if
param_def
and
not
isinstance
(value,
eval
(param_def.type)):
return
False
,
f
"Invalid type for
{name}
: expected
{param_def.type}
"
```

```
return
```

```
True
```

```
,
```

```
"""
```

总结：

内置插件包括Search、DALLE、Browser。每个插件遵循统一的BaseTool接口，参数定义明确，支持生命周期钩子。参数校验是安全的重要保障。

有问题欢迎留言。

相关系列阅读

Agent开发系列 14：内置工具实现

Agent开发系列 15：Function Calling + 工具安全

myAgent开发系列 09：插件架构设计

下期预告：第11篇《插件市场与发布》

myAgent开发系列 · 用代码理解AI

11. 插件市场与发布

myAgent开发系列 11

插件市场与发布

有了插件架构，接下来要做的是

让第三方能开发、发布、分享插件

——这就是插件市场。

(参考AI Coding系列17：前后端交互设计)

插件市场的核心是：注册、审核、发布、发现。

核心观点：

插件市场不是"把插件扔上去就行"，而是要解决：谁来审、怎么审、出了问题怎么办。这三个问题没想清楚，市场就是空中楼阁。

1

插件元数据

插件在市场上架，需要完整的元数据：

python

class

PluginManifest(BaseModel):

插件市场元数据

id: str

name: str

version: str

description: str

icon: str

分类和标签

category: str

search, image, productivity, etc.

tags: List[str]

作者信息

author: str

author_url: str

homepage: str

安全信息

permissions: List[str]

需要哪些权限

sandbox_mode: bool =

True

manifest.yaml 示例

name: my-plugin

version: 1.0.0

description: "A plugin that does X"

category: productivity

permissions: ["network"]

2

插件审核流程

插件上架前必须审核，防止恶意代码危害用户。

(详见Agent开发系列15：工具安全)

python

class

PluginReviewStatus:

PENDING =

"pending"

APPROVED =

"approved"

REJECTED =

"rejected"

REQUIRES_REVISION =

"requires_revision"

class

PluginReview:

plugin_id: str

version: str

status: str

审核内容

security_check: bool

安全扫描是否通过

functionality_check: bool

功能是否正常

documentation_check: bool

文档是否完整

审核结果

reviewer_comments: str

reviewed_at: datetime

reviewed_by: str

审核流程：

自动扫描

——代码静态分析、权限检查、恶意代码检测

功能测试

——沙箱环境运行，验证功能是否符合描述

人工审核

——审核员复核，特别复杂的插件需要专家评审

3

插件版本管理

插件也要有版本管理，支持升级和回滚：

python

class

PluginVersion:

plugin_id: str

version: str

语义化版本：major.minor.patch

changelog: str

兼容性

min_platform_version: str

最低平台版本要求

状态

status: str

draft, published, deprecated

统计

install_count: int =

0

rating: float =

0.0

4

插件搜索与发现

python

class

PluginSearchService:

async def

search

(self, query: str, filters: dict) -> List[PluginManifest]:

1. 全文搜索

results =

await

self.vector_search(query)

2. 过滤

if

filters.

get

(

"category"

):

results = [p

for

p

in

results

if

p.category == filters[

"category"

```
]]
```

```
# 3. 排序（综合评分）
```

```
results.
```

```
sort
```

```
(
```

```
key
```

```
lambda
```

```
p:
```

```
self
```

```
.
```

```
calculate_score
```

```
(p), reverse=
```

```
True
```

```
)
```

```
return
```

```
results[:20]
```

```
# 返回前20个
```

```
def
```

```
calculate_score
```

```
(self, plugin) -> float:
```

```
# 综合评分 = 安装量*0.3 + 评分*0.3 + 安全性*0.4
```

```
install_score =
```

```
min
```

```
(plugin.install_count /
```

```
10000
```

```
,
```

```
1.0
) *
0.3
rating_score = plugin.rating *
0.3
security_score =
0.4
if
plugin.security_check
else
0.0
return
install_score + rating_score + security_score
```

总结：

插件市场需要完整的元数据、审核流程、版本管理和搜索发现能力。审核是安全的重要保障，版本管理支持升级回滚，搜索发现让优质插件能被找到。

有问题欢迎留言。

相关系列阅读

AI Coding系列 17：前后端交互设计

Agent开发系列 15：工具安全

myAgent开发系列 10：内置插件实现

下期预告：第12篇《插件沙箱与安全》

myAgent开发系列 · 用代码理解AI

12. 插件沙箱与安全

myAgent开发系列 12

插件沙箱与安全

插件系统最核心的问题之一是：

安全。

(详见Agent开发系列15：工具安全)

用户上传的插件代码，可能是恶意的；第三方插件可能存在漏洞。沙箱隔离是保护系统安全的重要手段。

核心观点：

插件安全是生命线。永不要相信插件代码——再友好的插件，也要在沙箱里运行。资源限制、权限控制、代码审计，三管齐下才能保证安全。

1

资源限制：CPU、内存、网络

python

import

resource

import

multiprocessing

as

mp

class

ResourceLimiter:

@staticmethod

def

set_limits

():

内存限制：128MB

```
resource.setrlimit(resource.RLIMIT_AS, (
```

```
128
```

```
*
```

```
1024
```

```
*
```

```
1024
```

```
,
```

```
resource.RLIM_INFINITY
```

```
))
```

CPU时间限制：5秒

```
resource.setrlimit(resource.RLIMIT_CPU, (
```

```
5
```

```
,
```

```
5
```

```
))
```

禁止创建子进程

```
resource.setrlimit(resource.RLIMIT_NPROC, (
```

```
0
```

```
,
```

```
0
```

```
))
```

```
2
```

权限控制：声明式权限

python

```
class
```

```
Permission:
```

```
NETWORK =
```

```
"network"
```

```
# 允许网络请求
```

```
FILE_SYSTEM =
```

```
"file_system"
```

```
# 允许文件系统操作
```

```
EXTERNAL_API =
```

```
"external_api"
```

```
# 允许调用外部API
```

```
class
```

```
SandboxedExecutor:
```

```
@staticmethod
```

```
def
```

```
check_permission
```

```
(plugin_manifest, required: str) -> bool:
```

```
# 检查插件是否声明了该权限
```

```
return
```

```
required
```

```
in
```

```
plugin_manifest.permissions
```

```
async def
```

```
execute
```

```
(self, plugin: BaseTool, params: dict):
```

```
manifest = plugin.manifest
```

如果需要网络权限但没有声明，拒绝执行

if

not

self.

check_permission

(manifest, Permission.NETWORK):

拦截网络请求

params = self._sanitize_urls(params)

return

await

plugin.

execute

(**params)

3

代码审计：静态分析

插件上架前，必须经过静态代码扫描：

python

class

SecurityScanner:

DANGEROUS_PATTERNS = [

"eval("

,

"exec("

,

"__import__"

,

```
"subprocess"
```

```
,
```

```
"os.system"
```

```
,
```

```
"requests.get"
```

```
,
```

```
# 需要声明network权限
```

```
]
```

```
def
```

```
scan
```

```
(self, code: str) -> tuple[bool, list]:
```

```
issues = []
```

```
for
```

```
pattern
```

```
in
```

```
self.DANGEROUS_PATTERNS:
```

```
if
```

```
pattern
```

```
in
```

```
code:
```

```
issues.
```

```
append
```

```
(
```

```
f
```

```
"Dangerous pattern found:
```

```
{pattern}
```

```
"  
  
)  
  
# 检查是否尝试访问敏感API  
  
sensitive_apis = [  
  
    "os.environ"  
  
    ,  
  
    "os.getenv"  
  
    ,  
  
    "keyring"  
  
]  
  
for  
  
    api  
  
    in  
  
        sensitive_apis:  
  
            if  
  
                api  
  
                in  
  
                    code:  
  
                        issues.  
  
                            append  
  
                                (  
  
                                    f  
  
                                        "Sensitive API access:  
  
                                            {api}  
  
                                                "  
  
                                                    )
```

return

len

(issues) ==

0

, issues

4

运行时监控

python

class

PluginMonitor:

async def

execute_with_monitoring

(self, plugin: BaseTool, params: dict):

start_time = time.time()

start_memory = psutil.Process().memory_info().rss

try

:

result =

await

plugin.

execute

(**params)

status =

"success"

except

Exception

as

e:

result = ToolResult(success=

False

, error=

str

(e))

status =

"error"

finally

:

duration = time.time() - start_time

memory_used = psutil.Process().memory_info().rss - start_memory

记录监控数据

await

self.metrics.

record

(

plugin_id=plugin.manifest.id,

status=status,

duration=duration,

memory_used=memory_used

)

return

result

总结：

插件安全三板斧：资源限制（CPU/内存）、权限控制（声明式权限）、代码审计（静态分析）。运行时监控让你知道每个插件的实际表现，发现异常行为。

有问题欢迎留言。

相关系列阅读

Agent开发系列 15：工具安全

myAgent开发系列 11：插件市场与发布

下期预告：第13篇《插件调试与日志》

myAgent开发系列 · 用代码理解AI

13. 插件调试与日志

myAgent开发系列 13

插件调试与日志

插件写好了，怎么调试？线上出问题了，怎么追踪？

(详见Agent开发系列25：可观测性)

调试和日志是开发者和运维的"眼睛"，没有它们，就是在瞎子里走迷宫。

核心观点：

插件调试的核心是"可复现"——本地能跑，线上也能跑。日志的核心是"可追溯"——出问题能快速定位是哪个插件、哪次调用、哪个参数。

1

结构化日志

python

import

structlog

structlog.configure(

processors=[

structlog.processors.TimeStamper(fmt=

"iso"

),

structlog.processors.JSONRenderer()

]

)

logger = structlog.get_logger()

插件调用日志

```
logger.info(  
    "tool_called"  
  
    ,  
    tool_name=  
        "search"  
  
    ,  
    bot_id=  
        "bot_xxx"  
  
    ,  
    params={  
        "query"  
        :  
        "Python教程"  
    },  
    duration_ms=  
        234  
    )  
  
2
```

调用链路追踪

python

class

ToolCallTracer:

@contextmanager

def

trace

(self, tool_name: str, params: dict):

```
trace_id = uuid.uuid4().hex

start_time = time.time()

# 记录开始

logger.info(

"tool_start"

,

trace_id=trace_id,

tool_name=tool_name,

params=params

)

try

:

yield trace_id

finally

:

duration = (time.time() - start_time) *

1000

logger.info(

"tool_end"

,

trace_id=trace_id,

duration_ms=

int

(duration)

)

3
```

本地调试模式

python

class

LocalDebugger:

@staticmethod

def

debug_tool

(tool_class, params):

本地调试：不需要连接LLM，直接测试工具

tool = tool_class()

print(

f" Testing

{tool.definition.name}

"

)

print(

f" Definition:

{tool.definition.dict()}

"

)

print(

f" Params:

{params}

"

)

result =

asyncio

.

run

(tool.

execute

(**params))

print(

f" Result:

{result.dict()}

"

)

return

result

使用

LocalDebugger.

debug_tool

,

SearchTool,

{

"query"

:

"Python教程"

}

)

总结：

插件调试靠结构化日志、调用链路追踪、本地调试模式。日志要全链路可追溯，调试要能本地复现线上问题。

有问题欢迎留言。

相关系列阅读

Agent开发系列 25：可观测性

myAgent开发系列 12：插件沙箱与安全

下期预告：第14篇《 workflow架构设计》—— workflow引擎篇开始

myAgent开发系列 · 用代码理解AI